COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	Jul 16, 2026	**DATE SUBMITTED:**	Jul 21, 2026
NAME:	Carlo Mari B. Asajar	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 2.1 [ARRAY, POINTERS AND DYNAMIC MEMORY ALLOCATION]

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

In this activity, we will explore static and dynamic memory allocation through utilizing arrays and other components mentioned beforehand, this will be for the implementation of a student list with the students' names and age. To begin, we will include pertinent libraries for input/output stream and strings (which we will be using in this activity)

```
#include <iostream>
#include <string.h>
```

Now we will create the student class with private attributes studentName and studentAge.

```
class Student{
private:
	std::string studentName;
	int studentAge;
```

Then, as discussed beforehand, we have to define the constructor and the big three: the destructor, copy constructor and copy assignment operator. We will assign default values for the parameters of our constructor in the event that it is uninitialized.

```
public:
	//constructor
	Student(std::string newName = "John Doe", int newAge=18){
		studentName = std::move(newName);
		studentAge = newAge;
		std::cout << "Constructor Called." << std::endl;
	};
```

The big three is then defined. We will add an output stream for each to observe when each of the following functions are implicitly or explicitly called.

```
//deconstructor
	~Student(){
		std::cout << "Destructor Called." << std::endl;
	}

	//Copy Constructor
	Student(const Student &copyStudent) {
		std::cout << "Copy Constructor Called" << std::endl;
		studentName = copyStudent.studentName;
		studentAge = copyStudent.studentAge;
	}

	//Display Attributes
	void printDetails(){
		std::cout << this->studentName << " " << this->studentAge <<
std::endl;
	}
};
```

The driver program is now to be defined. We want to utilize the main function to simply show the static and dynamic allocation. However, we will explore the initial goal of creating instances of our Student class.

```
int main(){
	Student student1("Roman", 28);
	Student student2(student1);
	Student student3;
	student3 = student2;

	return 0;
}
```

Run the code and show the output. Include the screenshot in table 2-1 followed by your observation.
Now, modify the driver program so that we have the array size of 5, an array of Student objects, the list of students' names and their age.

```
int main(){
	const size_t j = 5;

	Student studentList[j] = {};
	std::string nameList[j] = {"Carly", "Freddy", "Sam", "Zack",
"Cody"};
	int ageList[j] = {15, 16, 18, 19, 16};

	return 0;
}
```

Run the code with the modified driver function and show the output. Include the screenshot in table 2-2 followed by your observation.

We have so far created static memory allocation through the use of the arrays. We will now dynamically allocate instances of the student class and store the newly created objects in the locations pointed to by our Array.

```
int main() {
	const size_t j = 5;

	Student studentList[j] = {};
	std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack",
"Cody"};
	int ageList[j] = {15, 16, 18, 19, 16};

	for(int i = 0; i < j; i++) { //loop A
		Student *ptr = new Student(namesList[i], ageList[i]);
		studentList[i] = *ptr;
	}

	for(int i = 0; i < j; i++) { //loop B
		studentList[i].printDetails();
	}

	return 0;
}
```

Discuss what is done by loop A and loop B in table 2-3. Additionally, discuss the output and whether the functions are working as intended. If any corrections were made, further provide your modification and analysis in table 2-4.

Screenshot

CPE010_HOA2p1_Asajar.cpp > main()

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
    //constructor
    Student(std::string newName = "John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };

    //deconstructor
    ~Student(){
        std::cout << "Destructor Called." << std::endl;
    }

    //Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    //Display Attributes
    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main(){
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

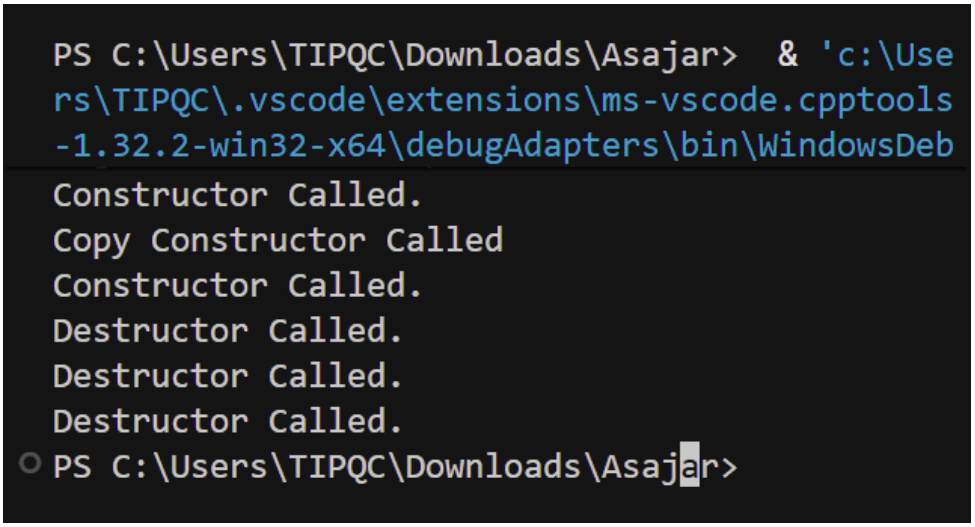
Output	
Observation	The `***Student student1("Roman", 28);***` calls the constructor and creates the first object with the given name and age. The line `***Student student2(student1);***` uses the copy constructor to make a copy of `***student1***`. Next, `***Student student3;***` calls the default constructor because no values are given, so it uses `***"John Doe"***` and `***18***`. The line `***student3 = student2;***` copies the values from `***student2***` to `***student3***` using the assignment operator, not the copy constructor, because `***student3***` already exists. When the `***main()***` function ends, the destructor is called for `***student3***`, `***student2***`, and `***student1***` in reverse order, showing how the program cleans up objects.

Table 2-1. Initial Driver Program

Screenshot

CPE010_HOA2p1_Asajar.cpp > main()

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
    //constructor
    Student(std::string newName = "John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };

    //deconstructor
    ~Student(){
        std::cout << "Destructor Called." << std::endl;
    }

    //Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    //Display Attributes
    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main(){
    const size_t j = 5;

    Student studentList[j] = {};
    std::string nameList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

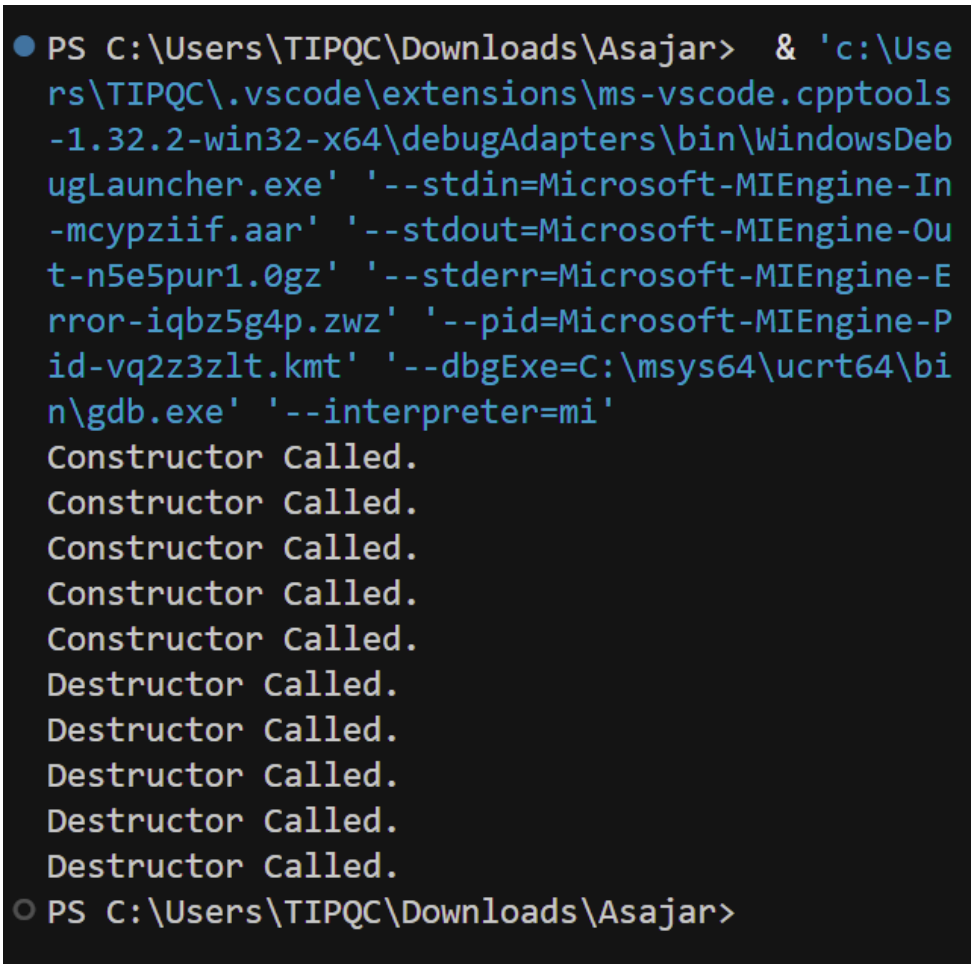
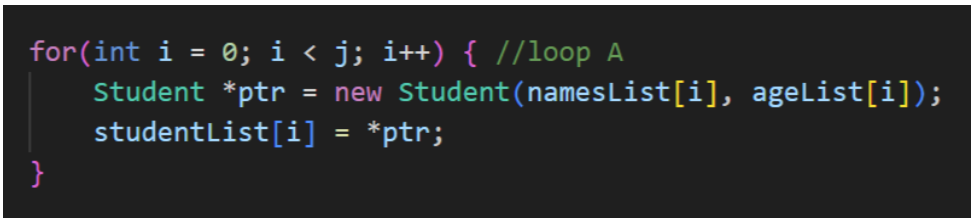
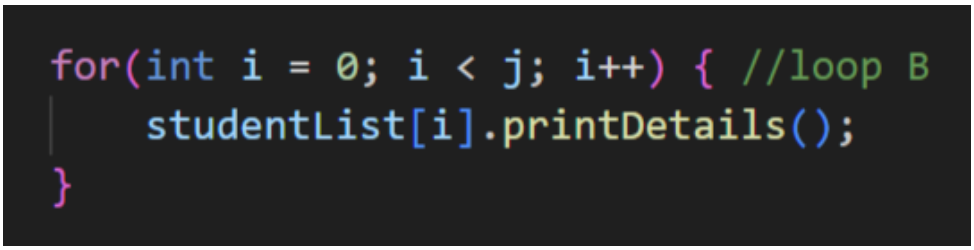
Output	
Observation	The `***const size_t j = 5;***` sets the size of the arrays to 5. The line `***Student studentList[j] = {};***` creates 5 `***Student***` objects, so the default constructor is called 5 times and each object gets the default values `***"John Doe"***` and `***18***`. The lines `***std::string nameList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};***` and `***int ageList[j] = {15, 16, 18, 19, 16};***` only store the names and ages in separate arrays. Since there is no code that assigns `***nameList***` and `***ageList***` to `***studentList***`, the `***Student***` objects keep their default values. When the `***main()***` function ends, the destructor is called 5 times because all the objects in `***studentList***` are destroyed automatically.

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	Loop A shows how the program uses `**new**`, pointers, and the assignment operator to put the student data into the `**studentList**` objects. The `**for(int i = 0; i < j; i++)**` in Loop A repeats the process 5 times to get all the student data from the arrays. The code `**Student *ptr = new Student(namesList[i], ageList[i]);**` creates a new `**Student**` object using the values from `**namesList**` and `**ageList**` based on the current index `**i**`. The code `**studentList[i] = *ptr;**` copies the created student's name and age into the matching position in the `**studentList**` array. Each loop changes the value of `**i**`, so it stores `**Carly**`, `**Freddy**`, `**Sam**`, `**Zack**`, and `**Cody**` with their correct ages.
Loop B	
Observation	Loop B uses the array index `**i**` and the function `**printDetails()**` to print all the student names and ages that were stored in the program. The `**for(int i = 0; i < j; i++)**` in Loop B repeats 5 times to access every student in the `**studentList**` array. The code `**studentList[i].printDetails();**` calls the `**printDetails()**` function for each object, starting from `**studentList[0]**` until `**studentList[4]**`. The `**printDetails()**` function uses `**this->studentName**` and `**this->studentAge**` inside the class to display the stored student information.

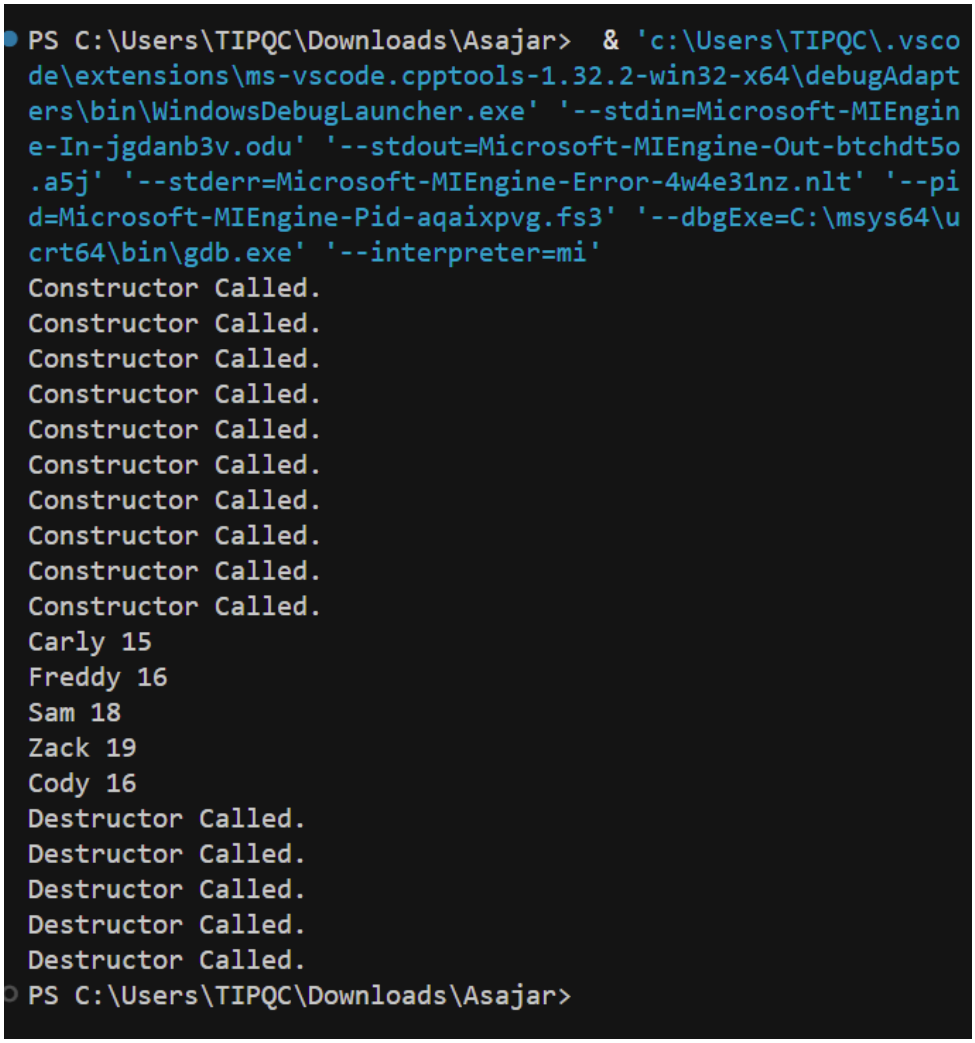
Output	
Observation	The final output shows the order of how the code creates objects, stores data using `***studentList[i] = *ptr***`, prints details, and destroys the objects. The final output starts with `***Constructor Called.***` because the code `***Student studentList[j] = {};***` and `***new Student(namesList[i], ageList[i]);***` create `***Student***` objects. The student names and ages are shown because Loop B uses the code `***studentList[i].printDetails();***` to display each student's information. The output `***Carly 15***`, `***Freddy 16***`, `***Sam 18***`, `***Zack 19***`, and `***Cody 16***` comes from the values inside `***namesList***` and `***ageList***` that are used in Loop A. The `***Destructor Called.***` messages appear after `***return 0;***` because the `***~Student()***` function runs when the `***studentList***` objects are destroyed.

Table 2-3. Final Driver Program

Modifications	**None**
Observation	**None**

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve Different Problems using the C++ Programming Language

Jenna's Grocery List		
Apple	PHP 10	x7
Banana	PHP 10	x8
Broccoli	PHP 60	x12
Lettuce	PHP 50	x10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

My Code:

CPE010_HOA2p1_Asajar_Supplementary.cpp > main()

```cpp
#include <iostream>
#include <iomanip>
#include <string>

class GroceryItem {
protected:
    std::string itemName;
    double unitPrice;
    int itemQuantity;

public:
    GroceryItem(const std::string& itemName = "", double unitPrice = 0, int itemQuantity = 0)
        : itemName(itemName), unitPrice(unitPrice), itemQuantity(itemQuantity) {}

    GroceryItem(const GroceryItem& other)
        : itemName(other.itemName),
          unitPrice(other.unitPrice),
          itemQuantity(other.itemQuantity) {}

    GroceryItem& operator=(const GroceryItem& other) {
        if (this != &other) {
            itemName = other.itemName;
            unitPrice = other.unitPrice;
            itemQuantity = other.itemQuantity;
        }
        return *this;
    }

    virtual ~GroceryItem() {}

    double calculateSum() const {
        return unitPrice * itemQuantity;
    }

    virtual std::string getType() const = 0;

    virtual void display() const {
        std::cout << std::left << std::setw(12) << getType()
                  << std::setw(15) << itemName << std::setw(10)
                  << ("PHP " + std::to_string((int)unitPrice))
                  << std::setw(8) << ("x" + std::to_string(itemQuantity))
                  << "PHP " << calculateSum() << std::endl;
    }
};

class Fruit : public GroceryItem {
public:
    Fruit(const std::string& itemName, double unitPrice, int itemQuantity)
        : GroceryItem(itemName, unitPrice, itemQuantity) {}

    Fruit(const Fruit& other)
        : GroceryItem(other) {}

    Fruit& operator=(const Fruit& other) {
        GroceryItem::operator=(other);
        return *this;
    }

```

```cpp
    std::string getType() const override {
        return "Fruit";
    }

    ~Fruit() override {
        std::cout << "Fruit \"" << itemName << "\" deleted." << std::endl;
    }
};

class Vegetable : public GroceryItem
{
public:
    Vegetable(const std::string& itemName, double unitPrice, int itemQuantity)
        : GroceryItem(itemName, unitPrice, itemQuantity) {}

    Vegetable(const Vegetable& other)
        : GroceryItem(other) {}

    Vegetable& operator=(const Vegetable& other) {
        GroceryItem::operator=(other);
        return *this;
    }

    std::string getType() const override {
        return "Vegetable";
    }

    ~Vegetable() override {
        std::cout << "Vegetable \"" << itemName << "\" deleted." << std::endl;
    }
};

double totalSum(GroceryItem* groceryList[], int listSize)
{
    double total = 0;

    for (int index = 0; index < listSize; index++) {
        if (groceryList[index] != nullptr) {
            total += groceryList[index]->calculateSum();
        }
    }

    return total;
}

void lines() {
    for (int i = 0; i < 61; i++) {
        std::cout << "-";
    }
    std::cout << std::endl;
}

void displayList(GroceryItem* groceryList[], int listSize) {
    lines();
    std::cout << "\t\t    JENNA'S GROCERY LIST" << std::endl;
    lines();

```

```
116     std::cout << std::left
117               << std::setw(12) << "Type" << std::setw(15) << "Item"
118               << std::setw(10) << "Price" << std::setw(8) << "Qty"
119               << "Subtotal" << std::fixed << std::setprecision(2) << std::endl;
120     lines();
121
122     for (int index = 0; index < listSize; index++) {
123         if (groceryList[index] != nullptr) {
124             groceryList[index]->display();
125         }
126     }
127
128     lines();
129     std::cout << "Total Amount: PHP "
130               << std::fixed << std::setprecision(2)
131               << totalSum(groceryList, listSize) << std::endl;
132     lines();
133     std::cout << std::endl;
134 }
135
136 int main() {
137     const int LIST_SIZE = 4;
138
139     GroceryItem* groceryList[LIST_SIZE];
140
141     groceryList[0] = new Fruit("Apple", 10, 7);
142     groceryList[1] = new Fruit("Banana", 10, 8);
143     groceryList[2] = new Vegetable("Broccoli", 60, 12);
144     groceryList[3] = new Vegetable("Lettuce", 50, 10);
145
146     std::cout << "================== INITIAL GROCERY LIST ==================="
147               << std::endl << std::endl;
148
149     displayList(groceryList, LIST_SIZE);
150
151     std::cout << "- Deleting Lettuce..." << std::endl;
152     std::cout << "- ";
153
154     delete groceryList[3];
155     groceryList[3] = nullptr;
156
157     std::cout << std::endl << std::endl
158               << "================== UPDATED GROCERY LIST ==================="
159               << std::endl << std::endl;
160
161     displayList(groceryList, LIST_SIZE);
162
163     for (int index = 0; index < LIST_SIZE - 1; index++) {
164         std::cout << "- ";
165         delete groceryList[index];
166         groceryList[index] = nullptr;
167     }
168     std::cout << std::endl;
169
170     return 0;
171 }
```

My Output:

```
PS C:\Users\carlo\Downloads\ASAJAR_CPE 010_Hands-on Activity 2.1_Arrays,
Pointers and Dynamic Memory Allocation>  & 'c:\Users\carlo\.vscode\extens
ions\ms-vscode.cpptools-1.32.2-win32-x64\debugAdapters\bin\WindowsDebugLa
uncher.exe' '--stdin=Microsoft-MIEngine-In-5h24c1wq.xr3' '--stdout=Micros
oft-MIEngine-Out-2xal0isz.rty' '--stderr=Microsoft-MIEngine-Error-detpkgq
5.ypa' '--pid=Microsoft-MIEngine-Pid-5zlrineg.ne1' '--dbgExe=C:\msys64\uc
rt64\bin\gdb.exe' '--interpreter=mi'
================== INITIAL GROCERY LIST ====================

------------------------------------------------------------
                    JENNA'S GROCERY LIST
------------------------------------------------------------
Type        Item          Price     Qty     Subtotal
------------------------------------------------------------
Fruit       Apple         PHP 10    x7      PHP 70.00
Fruit       Banana        PHP 10    x8      PHP 80.00
Vegetable   Broccoli      PHP 60    x12     PHP 720.00
Vegetable   Lettuce       PHP 50    x10     PHP 500.00
------------------------------------------------------------
Total Amount: PHP 1370.00
------------------------------------------------------------

- Deleting Lettuce...
- Vegetable "Lettuce" deleted.


================== UPDATED GROCERY LIST ====================

------------------------------------------------------------
                    JENNA'S GROCERY LIST
------------------------------------------------------------
Type        Item          Price     Qty     Subtotal
------------------------------------------------------------
Fruit       Apple         PHP 10    x7      PHP 70.00
Fruit       Banana        PHP 10    x8      PHP 80.00
Vegetable   Broccoli      PHP 60    x12     PHP 720.00
------------------------------------------------------------
Total Amount: PHP 870.00
------------------------------------------------------------

- Fruit "Apple" deleted.
- Fruit "Banana" deleted.
- Vegetable "Broccoli" deleted.
```

Code Analysis:

The program was created to manage Jenna's grocery list by using C++ classes, inheritance, and dynamic memory allocation. The code starts by creating a base class called ***GroceryItem*** that contains the important attributes such as ***itemName***, ***unitPrice***, and ***itemQuantity***, which are used to store the details of each grocery item. The constructor, copy constructor, copy assignment operator, and virtual destructor inside the class were added to properly handle object creation, copying, updating, and deleting. The ***calculateSum()*** function was important because it calculates the subtotal of each item by multiplying the ***unitPrice*** and ***itemQuantity***.

The ***Fruit*** and ***Vegetable*** classes were created from the ***GroceryItem*** class using inheritance, where the ***getType()*** function helps identify if the item belongs to the fruit or vegetable category. The ***display()*** function was also important because it shows the details of every item including the type, name, price, quantity, and subtotal in an organized format. The ***totalSum()*** function was created to calculate the overall amount that needs to be paid by adding all the subtotal values from the ***groceryList*** array. The ***displayList()*** function was used to arrange the output properly by using ***setw()*** from the ***<iomanip>*** library, making the information easier to read.

The code used an array of pointers called ***groceryList*** to store different items like ***Apple***, ***Banana***, ***Broccoli***, and ***Lettuce***, allowing each item to be accessed and displayed correctly. The use of ***new*** and ***delete*** was also an important part of the code because it manages the memory assigned to each grocery item, especially when deleting the ***Lettuce*** object and removing its memory allocation. The virtual destructor in the ***GroceryItem*** class was needed to make sure that the correct destructor from the child classes will run when deleting objects through the base class pointer.

Some difficulties faced while creating the code were understanding how inheritance connects the ***GroceryItem***, ***Fruit***, and ***Vegetable*** classes because each class needs to work together properly. Another difficulty was creating the copy constructor and copy assignment operator since the values of ***itemName***, ***unitPrice***, and ***itemQuantity*** must be copied correctly. Handling dynamic memory was also challenging because the use of ***new*** and ***delete*** needs to be done carefully to prevent memory problems. Formatting the output using ***setw()***, ***fixed***, and ***setprecision()*** was also a little difficult because small changes can affect the arrangement of the displayed grocery list.

Conclusion:

The activity was able to apply different concepts such as the memory allocation, objects, classes, and how different programming concepts work together. From the procedure, the activity showed the importance of static and dynamic memory allocation, especially when using arrays, pointers, and dynamically created objects. The process of creating the Student class helped in understanding how constructors, copy constructors, copy assignment operators, and destructors work when objects are created, copied, and removed. It also showed how pointers and the use of new and delete can manage memory properly. While doing the procedure, some difficulties were faced because understanding the flow of memory and how objects are stored was not easy at first. The use of pointers was also challenging because small mistakes in handling memory can affect the whole program. However, by testing the outputs and observing how each function works, the concepts became more understandable.

For the supplementary activity, the grocery list program helped apply the lessons learned into a bigger and more practical situation. The activity used classes, inheritance, dynamic memory allocation, arrays of pointers, and object management to create a program that can store and calculate grocery items. Creating the GroceryItem, Fruit, and Vegetable classes helped show how inheritance allows different classes to share common functions while still having their own purpose. The use of functions like calculateSum(), display(), and totalSum() also helped in organizing the code and making the program easier to manage. The deletion of the Lettuce object using delete also gave a better understanding of how dynamic memory should be handled properly. The challenging parts of the supplementary activity were connecting all the classes together, making sure the copy constructor and copy assignment operator worked correctly, and preventing possible memory errors. Formatting the output using setw(), fixed, and setprecision() was also difficult because small changes in the code can affect how the results appear.

The activity was a good experience because it gave a better understanding of how C++ works beyond simple coding. The activity was not completely easy because there were many new concepts that needed more practice, especially pointers, inheritance, and memory management. However, after checking the output and fixing some problems, the program became more understandable and the purpose of each part of the code became clearer. The performance in this activity was good because the required concepts were successfully applied, but there are still areas that need improvement such as writing cleaner code, understanding memory allocation faster, and becoming more confident when using advanced C++ features.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **formatting, explanation of concepts, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain how the code works by giving a detailed explanation on the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.